

MODUL PRAKTIKUM SISTEM INFORMASI GEOGRAFIS

Database & PostGIS (Backend Spasial)

Project webgis

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

- Mengaktifkan ekstensi PostGIS di Supabase
- Membuat struktur database untuk WebGIS pengelolaan sampah
- Menambahkan kolom spasial (geometry/geography)
- Mengatur relasi antar tabel
- Mengimplementasikan Row Level Security (RLS)
- Membuat policies berbasis JWT untuk keamanan data

2. Alat dan Tools yang Digunakan

- Browser (Chrome / Edge / Firefox)
- Supabase
- Visual Studio Code
- React
- Leaflet
- Node.js & npm
- Koneksi Internet

3. Materi Singkat

3.1 Apa itu PostGIS?

PostGIS adalah ekstensi dari PostgreSQL yang memungkinkan penyimpanan dan pengolahan data spasial seperti:

- Titik (Point) → lokasi warga
- Garis (Line) → jalur pengangkutan
- Area (Polygon) → wilayah

3.2 Konsep Data Spasial

Jenis kolom spasial:

- **geometry** → fleksibel, umum untuk GIS
- **geography** → lebih akurat untuk koordinat bumi (lat/lng)

Contoh:

`location geography(Point, 4326)`

3.3 Relasi Database

Relasi digunakan untuk menghubungkan tabel:

- One to Many
- Foreign Key

Contoh:

- warga → sampah
- warga → pembayaran

3.4 Row Level Security (RLS) & JWT

- RLS = pembatasan akses data per user
- JWT digunakan untuk identifikasi user login

4. Langkah-Langkah Praktikum

4.1 Aktifkan Extension

```
SQL Enable UUID Generation Extension × +  
1 create extension if not exists "uuid-oss";
```

```
SQL Enable PostGIS Extension +  
1 create extension if not exists postgis;
```

4.2 Membuat Struktur Database (Sesuai ERD)

Tabel Profiles

```
SQL User Profiles Table +  
1 create table profiles (  
2   id uuid primary key references auth.users(id),  
3   nama text,  
4   role text check (role in ('admin','warga','transporter')),  
5   created_at timestamp default now()  
6 );
```

Tabel Warga

```
create table warga (  
  id uuid default uuid_generate_v4() primary key,  
  user_id uuid references profiles(id),  
  nama text,  
  alamat text,  
  location geography(Point, 4326)  
);
```

Tabel Sampah

```
+  
1 create table sampah (  
2   id uuid default uuid_generate_v4() primary key,  
3   warga_id uuid references warga(id),  
4   jenis text,  
5   berat numeric,  
6   status text default 'menunggu'  
7 );
```

Tabel Pengangkutan

```
SQL Pengangkutan Status Tracking +
1 create table pengangkutan (
2   id uuid default uuid_generate_v4() primary key,
3   warga_id uuid references warga(id),
4   transporter_id uuid references profiles(id),
5   status text default 'menunggu'
6 );
```

Tabel Pembayaran

```
SQL Pembayaran Tracking Table +
1 create table pembayaran (
2   id uuid default uuid_generate_v4() primary key,
3   warga_id uuid references warga(id),
4   status text default 'belum',
5   tanggal timestamp
6 );
```

5. Row Level Security (RLS)

5.1 Aktifkan RLS

```
1 alter table warga enable row level security;
```

```
1 alter table sampah enable row level security;
```

```
1 alter table pengangkutan enable row level security;
```

```
1 alter table pembayaran enable row level security;
```

```
1 alter table profiles enable row level security;
```

5.2 Buat Fungsi Pengecek Role

Fungsi ini akan mengambil role dari tabel `profiles` secara aman.

Fungsi untuk mengecek role user saat ini

```
SQL Retrieve Current User Role +
1 create or replace function get_my_role()
2 returns text as $$
3 | select role from public.profiles where id = auth.uid();
4 $$ language sql security definer;
```

5.3. POLICY TABEL PROFILES

- Warga/Transporter: Bisa melihat dan mengubah profil sendiri.
- Admin: Bisa melihat semua profil.
- Public (Register): Bisa memasukkan data saat daftar.

Izin Melihat: Milik sendiri atau Admin

```
1 create policy "profiles_select" on profiles for select
2 using ( auth.uid() = id or get_my_role() = 'admin' );
```

Izin Insert: Untuk pendaftaran (milik sendiri)

```
1 create policy "profiles_insert" on profiles for insert
2 with check ( auth.uid() = id );
```

Izin Update: Milik sendiri atau Admin

```
1 create policy "profiles_update" on profiles for update
2 using ( auth.uid() = id or get_my_role() = 'admin' );
```

5.4. POLICY TABEL WARGA

- Warga: Bisa melihat/mengubah data sendiri.
- Transporter: Bisa melihat semua data (untuk titik di peta).
- Admin: Akses penuh.

Izin Melihat: Milik sendiri, Transporter, atau Admin

```
1 create policy "warga_select" on warga for select
2 using ( auth.uid() = user_id or get_my_role() in ('admin', 'transporter') );
```

Izin Insert: Warga (milik sendiri)

```
1 create policy "warga_insert" on warga for insert
2 with check ( auth.uid() = user_id );
```

Izin Update/Delete: Milik sendiri atau Admin

```
SQL Row-Level Access Policy for warga +
1 create policy "warga_all" on warga for all
2 using ( auth.uid() = user_id or get_my_role() = 'admin' );
```

5.5. POLICY TABEL `PEMBAYARAN` & `PENGANGKUTAN` & `SAMPAH`

Logikanya mirip: Warga hanya bisa melihat datanya sendiri (melalui `warga\_id`), sedangkan Admin dan Transporter memiliki akses sesuai tugasnya.

Untuk Tabel `pembayaran`:

```
SQL Row-Level Select Access for Pembayaran +
1 create policy "pembayaran_select" on pembayaran for select
2 using ( exists (select 1 from warga where id = warga_id and user_id = auth.uid()) or get_my_role() = 'admin' );
```

```
SQL Row-Level Security for Pembayaran Inserts +
1 create policy "pembayaran_insert" on pembayaran for insert
2 with check ( exists (select 1 from warga where id = warga_id and user_id = auth.uid()) );
```

```
SQL Admin Policy for Pembayaran Access +
1 create policy "pembayaran_admin" on pembayaran for all
2 using ( get_my_role() = 'admin' );
```

Untuk Tabel `pengangkutan`:

```
SQL Row-Level Security for Pengangkutan Select +
1 create policy "pengangkutan_select" on pengangkutan for select
2 using ( exists (select 1 from warga where id = warga_id and user_id = auth.uid()) or
get_my_role() in ('admin', 'transporter') );
```

Buat kebijakan baru yang mengizinkan Warga (untuk request) DAN Transporter (untuk mengambil tugas)

```
SQL Access Control for Pengangkutan Inserts x +
1 create policy "pengangkutan_insert" on pengangkutan for insert
2 with check (
3 (exists (select 1 from warga where id = warga_id and user_id = auth.uid())) -- Jika dia
Warga-nya
4 or
5 (get_my_role() = 'transporter') -- Jika dia Transporter
6 );
7 |
```

Pastikan Transporter juga bisa mengupdate status tugas yang mereka ambil

drop policy if exists "pengangkutan_update" on pengangkutan;

```
SQL Update Access Control for Pengangkutan +
1 create policy "pengangkutan_update" on pengangkutan for update
2 using (
3   get_my_role() = 'admin'
4   or
5   (get_my_role() = 'transporter' and transporter_id = auth.uid())
6 );
```

Admin bisa hapus

```
SQL Delete Policy for Pengangkutan +
1 create policy "pengangkutan_delete" on pengangkutan for delete
2 using ( get_my_role() = 'admin' );
```

Untuk Tabel `sampah`:

```
SQL Row-Level Policy for sampah Access +
1 create policy "sampah_warga" on sampah for all
2 using ( exists (select 1 from warga where id = warga_id and user_id = auth.uid()) );
```

```
SQL Admin-Only Select Policy +
1 create policy "sampah_admin" on sampah for select
2 using ( get_my_role() = 'admin' );
```

Kesimpulan Pengaturan POLICY:

1. Warga: Hanya bisa mengelola data yang terhubung dengan `user\_id` mereka sendiri.
2. Transporter: Bisa melihat data `warga` (untuk peta) dan mengupdate status `pengangkutan`.
3. Admin: Memiliki izin bypass melalui pengecekan `get\_my\_role() = 'admin'`.

6. Matikan Konfirmasi Email (Sementara)

Di bagian *Auth Providers > Email*, nonaktifkan "Confirm email" untuk menghindari pengiriman email verifikasi ganda saat testing.



Confirm email
Users will need to confirm their email address before signing in for the first time

6. Tugas Praktikum

Kerjakan tugas berikut:

1. Aktifkan PostGIS di Supabase
2. Buat semua tabel:
 - warga
 - sampah
 - pengangkutan
 - pembayaran
 - profiles
3. Buat relasi antar tabel
4. Aktifkan RLS
5. Buat policy

Screenshot:

- Extension PostGIS aktif
- Struktur tabel (Schema Visualizer)
- RLS & Policies

Simpan dalam laporan (Word/PDF)